

EECE 7205: Fundamentals of Computer Engineering

Fall 2026

Assignment #01: Sorting Techniques and Interview Problems

Student Name	Catalina Buitano
NUID	NUID 002330106
Email	buitano.c@northeastern.edu
Instructor	Dr. Fatema Nafa
Due Date	September 2026

Contents

Problem 1: Merge Intervals	2
Problem 2: Sort Colors	6
Problem 3: Largest Number	11
Problem 4: Group Anagrams	15
Problem 5: Meeting Rooms	18
Problem 6: K Closest Points to the Origin	21
Problem 7: Top K Frequent Values	27
Problem 8: Merge K Sorted Arrays	32
Problem 9: Kth Largest Element	37
Problem 10: Count Inversions	42

Problem 1: Merge Intervals

1. Understanding the Problem

Description: Intervals appear in calendars, resource reservations, timelines, and scheduling systems. Overlapping intervals can be combined into a smaller set of non-overlapping intervals.

Question: Given an array of intervals $[start, end]$, sort the intervals by starting value and merge every overlapping interval. Return the resulting non-overlapping intervals.

Goal: The goal here is to merge all overlapping intervals in a given list so that the result is the smallest set of non-overlapping intervals.

Inputs: The inputs here are an array of intervals, represented as a pair $[start, end]$, which is not necessarily in sorted order. In the program, the user enters the number of intervals and then the start and end of each interval.

Outputs: New array of intervals, which are sorted by start value, in which no two intervals overlap.

Constraints: $start \leq end$ for every interval.

Assumptions: Intervals that touch exactly at an endpoint, where one ends where the next begins, are treated as overlapping and need to be merged.

Edge Cases: Here, some edge cases could be an empty input list, a list with only one interval, no intervals overlap at all, or all intervals overlap into a single merged interval.

2. Designing the Solution

So in order to design my solution, I first needed to visualize it. I drew out a line of intervals and used sample interval pairs. For example, inputs $[1, 3]$, $[2, 6]$, $[8, 10]$. When a human is doing this math, we can think of this as being like blocks on a timeline, as we are approximating which ‘blocks’ or in our case, interval pairs, overlap. $[1, 3] + [2, 6]$ overlap because the second pair begins with 2, while the first does not end until 3. This becomes $[1, 6]$. $[8, 10]$ does not overlap with $[1, 6]$, so we leave it alone and we get $[1, 6]$ and $[8, 10]$. Getting the program to recognize this overlap works a bit differently than it would for a human. In this case, we first sort the intervals according to their starting values. We then add the first interval to a result vector. Then we examine each remaining interval from left to right, comparing it with the end value of the last interval stored in the result. If the current starting value is less than or equal to the previous ending value, the intervals overlap and should be merged, and the ending value becomes the maximum of the two ending values. Otherwise, the intervals do not overlap, so the current interval is added to the result.

3. Pseudocode

Algorithm 1 Merge Intervals

```
1: function MERGEINTERVALS(intervals)
2:   if intervals is empty then
3:     return empty list
4:   end if
5:   sort intervals by starting value
6:   merged  $\leftarrow$  list containing the first interval
7:   for each interval current starting from the second interval do
8:     last  $\leftarrow$  last interval in merged
9:     if current.start  $\leq$  last.end then
10:      last.end  $\leftarrow$  max(last.end, current.end)
11:    else
12:      add current to merged
13:    end if
14:  end for
15:  return merged
16: end function
```

4. Correctness Explanation

Because intervals are sorted by start value before merging, in a sort and sweep algorithm, any interval that is capable of overlapping with the interval currently being built must appear immediately next in sorted order. Comparing each new interval only against the most recently merged interval is therefore sufficient to catch every overlap. For duplicate intervals, duplicates satisfy the overlap condition and merge into the existing interval without changing the result or breaking. Taking the maximum end value when merging ensures the wider interval is retained, regardless of which tied interval was processed initially. For empty inputs, this is handled at the start of the algorithm, which will return an empty list.

5. Time and Space Complexity

The time complexity here would be $O(n \log n)$ in all best, average, and worst case scenarios. The sort dominates because comparison sorting would not drop below $O(n \log n)$ even for nearly sorted input. The space complexity is $O(n)$ for the sorted copy and the result list.

6. C++ Implementation

```
// PROBLEM 1: Merge Intervals
vector<vector<int>> mergeIntervals(vector<vector<int>> intervals) {
    if (intervals.empty()) return {};

    sort(intervals.begin(), intervals.end(),
        [](const vector<int>& a, const vector<int>& b) {
            return a[0] < b[0];
        });

    vector<vector<int>> result;
```

```

result.push_back(intervals[0]);

for (size_t i = 1; i < intervals.size(); i++) {
    vector<int>& last = result.back();
    vector<int>& current = intervals[i];

    if (current[0] <= last[1]) {
        last[1] = max(last[1], current[1]);
    } else {
        result.push_back(current);
    }
}
return result;
}

```

7. Test Cases

Test	Input	Expected Output
1 (given example)	[1, 3] [2, 6] [8, 10] [15, 18]	[1, 6] [8, 10] [15, 18]
2 (edge case: touching intervals)	[1, 4] [4, 5]	[1, 5]
3 (edge case: empty input)	(none)	(empty)

8. Execution Screenshots

```

Enter start and end for interval 4: 15 18
Merged intervals: [1,6] [8,10] [15,18]

==== EECE 7205 Assignment 1 Menu ====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points to Origin (Sort vs Heap)
7. K Most Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element (Quickselect)
10. Count Inversions
0. Exit
Enter your choice: 

```

Figure 1: Test Case 1: Merge Intervals

```
Enter start and end for interval 2: 4 5
Merged intervals: [1,5]

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points to Origin (Sort vs Heap)
7. K Most Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element (Quickselect)
10. Count Inversions
0. Exit
Enter your choice: █
```

Figure 2: Test Case 2: Touching Intervals

```
Enter number of intervals: 0
Merged intervals:

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points to Origin (Sort vs Heap)
7. K Most Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element (Quickselect)
10. Count Inversions
0. Exit
Enter your choice: █
```

Figure 3: Test Edge Case 3: Single Element

Problem 2: Sort Colors

1. Understanding the Problem

Description: A manufacturing line labels items with three categories represented by 0, 1, and 2. The items must be grouped in category order. Given an array containing only 0, 1, and 2, sort it in place. First implement a counting-based solution. Then implement the one-pass Dutch National Flag algorithm without using `std::sort()`.

Goal: The goal here is to sort an array that contains only the values 0, 1, & 2 in place, so that all 0s come first, then all 1s, and then 2s, in numerical order.

Inputs: The inputs here are the values/integers where every value is either 0, 1, or 2. There could be one of each or multiple, but they must still be sorted in numerical order.

Outputs: The output is the same array, but reorganized in place so that it is sorted in categorical order. The program prints the result of both the counting sort and the Dutch National Flag algorithm.

Constraints: Every element is guaranteed to be exactly 0, 1, or 2, so no other values appear. The sort must be done in place (no `std::sort()` is allowed).

Assumptions: One assumption is that the array can contain any number of each category, including zero instances of one or more categories of the 3 numbers.

Edge Cases:

1. Empty array
2. Array with one element
3. Array containing only one distinct value
4. Array is sorted already
5. Array is in reverse order

2. Designing the Solution

I used two required implementations. The first is counting sort, which counts how many 0s, 1s, and 2s appear in the array, then overwrites the array by writing that many 0s, followed by that many 1s, followed by that many 2s. The second is the Dutch National Flag, also known as one pass in place. This approach uses three pointers, *low*, *mid*, and *high*, to split the array into 3 regions within a single pass, swapping elements into place as the pointer *mid* scans through the array. In this case, counting sort works well because there are only three possible values, so counting occurrences is fast and efficient. The Dutch National Flag algorithm is the more efficient, single pass alternative that avoids a second pass to write the array back. It can execute in place partitioning without relying on a general purpose sort.

3. Pseudocode

Counting Solution:

Algorithm 2 Counting Sort Colors

```
1: function COUNTINGSORTCOLORS(arr)
2:   count[0], count[1], count[2]  $\leftarrow$  0
3:   for each value v in arr do
4:     count[v]  $\leftarrow$  count[v] + 1
5:   end for
6:   index  $\leftarrow$  0
7:   for c  $\leftarrow$  0 to 2 do
8:     for i  $\leftarrow$  1 to count[c] do
9:       arr[index]  $\leftarrow$  c
10:      index  $\leftarrow$  index + 1
11:    end for
12:  end for
13:  return arr
14: end function
```

DNF Solution:

Algorithm 3 Dutch National Flag Sort

```
1: function DUTCHFLAGSORT(arr)
2:   low  $\leftarrow$  0
3:   mid  $\leftarrow$  0
4:   high  $\leftarrow$  LENGTH(arr) - 1
5:   while mid  $\leq$  high do
6:     if arr[mid] = 0 then
7:       swap arr[low] and arr[mid]
8:       low  $\leftarrow$  low + 1
9:       mid  $\leftarrow$  mid + 1
10:    else if arr[mid] = 1 then
11:      mid  $\leftarrow$  mid + 1
12:    else
13:      swap arr[mid] and arr[high]
14:      high  $\leftarrow$  high - 1
15:    end if
16:  end while
17:  return arr
18: end function
```

4. Correctness Explanation

In counting sort, since every value is known to be 0, 1, or 2, counting the occurrences of each and rewriting the array in that fixed order guarantees a correctly sorted result. In the DNF approach, the algorithm keeps three invariants at every step. Everything before *low* is 0, everything between *low* and *mid* is 1, and everything after *high* is 2. The region between *mid* and *high* is not yet examined. Each iteration extends one of the 0, 1, or 2 regions and shrinks the unexamined region by one element, and the loop ends after *mid* passes *high*, which means every element has been placed in its correct region. In terms of addressing the edge cases, duplicate values are tolerated by both

algorithms. Counting sort counts all instances and the DNF algorithm processes each element once regardless of whether or not it repeats. For empty input, both algorithms can tolerate an empty array correctly. Counting sort counts nothing and returns nothing back. DNF's loop condition, *mid* is less than or equal to *high*, is immediately false when the array is empty, so the loop exits right away.

5. Time and Space Complexity

For counting sort the time complexity is $O(n)$, since it requires one pass to count and one pass to rewrite the array. This is the same in the best, average, and worst case. The space complexity is $O(1)$ additional space because only a fixed size count array of length 3 is being used. For DNF, the time complexity is $O(n)$, since each element is visited a constant number of times as *mid* advances through the array exactly once (the *high* pointer only moves inward). This holds in the best, average, and worst case. The space complexity is $O(1)$ additional space, since sorting is done in place using only three pointers.

6. C++ Implementation

```
1 // PROBLEM 2: Sort Colors (Counting Sort + Dutch National Flag)
2 void countingSortColors(vector<int>& arr) {
3     int count[3] = {0, 0, 0};
4
5     for (int v : arr) {
6         count[v]++;
7     }
8
9     int index = 0;
10    for (int c = 0; c <= 2; c++) {
11        for (int i = 0; i < count[c]; i++) {
12            arr[index++] = c;
13        }
14    }
15 }
16
17 void dutchFlagSort(vector<int>& arr) {
18     int low = 0;
19     int mid = 0;
20     int high = (int)arr.size() - 1;
21
22     while (mid <= high) {
23         if (arr[mid] == 0) {
24             swap(arr[low], arr[mid]);
25             low++;
26             mid++;
27         } else if (arr[mid] == 1) {
28             mid++;
29         } else {
30             swap(arr[mid], arr[high]);
31             high--;
32         }
33     }
34 }
35
36 void runProblem2() {
37     int n;
```

```

38     cout << "Enter number of elements: ";
39     cin >> n;
40
41     vector<int> arr(n);
42     cout << "Enter " << n << " values (only 0, 1, or 2): ";
43     for (int i = 0; i < n; i++) {
44         cin >> arr[i];
45     }
46
47     vector<int> arrCopy = arr;
48
49     countingSortColors(arr);
50     cout << "Counting sort result: ";
51     for (int v : arr) cout << v << " ";
52     cout << endl;
53
54     dutchFlagSort(arrCopy);
55     cout << "Dutch National Flag result: ";
56     for (int v : arrCopy) cout << v << " ";
57     cout << endl;
58 }

```

7. Test Cases

Test	Input	Expected Output
1 (given example)	2 0 2 1 1 0	0 0 1 1 2 2
2 (edge case: single element)	0	0
3 (edge case: empty array)	(none)	(empty)

8. Execution Screenshots

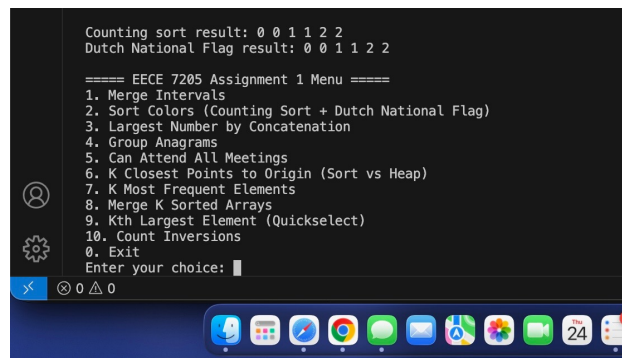


Figure 4: Test Case 1



Figure 5: Test Case 2: Single Element

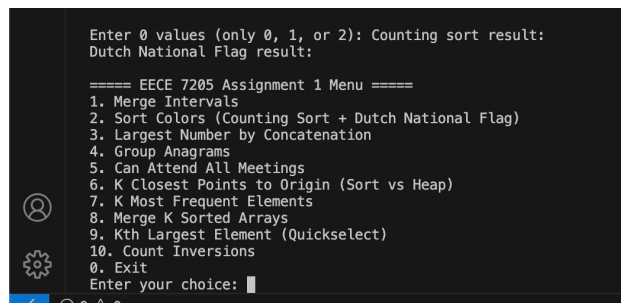


Figure 6: Test Edge Case 3: No Array

Problem 3: Largest Number

1. Understanding the Problem

Description: A custom ordering is sometimes required when the normal numeric order does not produce the desired combined result. Given a vector of nonnegative integers, arrange them so that their concatenation forms the largest possible number. Return the answer as a string.

Goal: The goal here is to arrange a list of nonnegative integers so that concatenating them together in the selected order produces the biggest possible resulting value, which is then returned as a string.

Input: Vector of nonnegative integers.

Output: A string representing the largest number formed by concatenating all the input integers in the best order.

Constraints: One constraint is that all integers are nonnegative, and integers may have differing numbers of digits.

Assumptions: One assumption is that the result may contain leading zeros only when every input number is 0, in which case the answer is "0".

Edge Cases:

1. Single element list
2. All elements are 0
3. Numbers with different digit lengths

2. Designing the Solution

Here the approach needed is converting each number to a string, then sorting the strings using a custom comparator: string a should come before string b if $a + b > b + a$. After sorting, concatenate all strings in that order. This strategy is appropriate here because comparing numbers by their normal numeric value doesn't work, because the "best" order depends on how digits combine when concatenated, not on the numbers' individual sizes. Comparing the two possible concatenations defines a valid total ordering, so a standard comparison sort can be utilized.

3. Pseudocode

Algorithm 4 Largest Number

```
1: function LARGESTNUMBER(nums)
2:   strs  $\leftarrow$  empty list
3:   for each number num in nums do
4:     add num as a string to strs
5:   end for
6:   sort strs so that a comes before b if  $a + b > b + a$ 
7:   result  $\leftarrow$  empty string
8:   for each string s in strs do
9:     result  $\leftarrow$  result + s
10:  end for
11:  if result is not empty and result starts with '0' then
12:    return "0"
13:  end if
14:  return result
15: end function
```

4. Correctness Explanation

The comparator $a + b > b + a$ directly compares the two possible outcomes of placing a before b versus b before a , so sorting by this rule always places each pair of elements in the order that produces a larger combined string. Applying it across the whole list using standard sort results in an optimal ordering. Duplicate values compare as equal under the comparator, so their relative order doesn't affect the result and they can be placed in either order without changing the final output. If the result starts with '0', every input must be 0, since any non-zero number would have been sorted ahead of it, so the program returns "0" instead of a string like "00". Values that produce the same result in either order, for example, "12" and "21", are treated as equal by the comparator, so there's no tie breaking logic needed in this case.

5. Time and Space Complexity

Time complexity is $O(n \log n \cdot k)$, where n is the number of integers and k is the average number of digits per number. The sort performs $O(n \log n)$ comparisons, and each comparison takes $O(k)$ time. This is true across the best, average, and worst cases, since the sort's comparison count does not change based on input order. Space complexity is $O(n \cdot k)$, for storing the string representations of all numbers and the final result string.

6. C++ Implementation

```
1 string largestNumber(vector<int>& nums) {
2     vector<string> strs;
3     for (int num : nums) {
4         strs.push_back(to_string(num));
5     }
6
7     sort(strs.begin(), strs.end(), [](const string& a, const string& b) {
8         return (a + b) > (b + a);
9     });
10    return strs.empty() ? "0" : join(strs, "");
11 }
```

```

9     });
10
11     string result;
12     for (const string& s : strs) {
13         result += s;
14     }
15
16     if (!result.empty() && result[0] == '0') {
17         return "0";
18     }
19     return result;
20 }
21
22 void runProblem3() {
23     int n;
24     cout << "Enter number of elements: ";
25     cin >> n;
26
27     vector<int> nums(n);
28     cout << "Enter " << n << " nonnegative integers: ";
29     for (int i = 0; i < n; i++) {
30         cin >> nums[i];
31     }
32
33     string result = largestNumber(nums);
34     cout << "Largest number: " << result << endl;
35 }

```

7. Test Cases

Test	Input	Expected Output
1 (given example)	3 30 34 5 9	9534330
2 (edge case: all zeros)	0 0	0
3 (edge case: different digit lengths)	10 2	210

8. Execution Screenshots

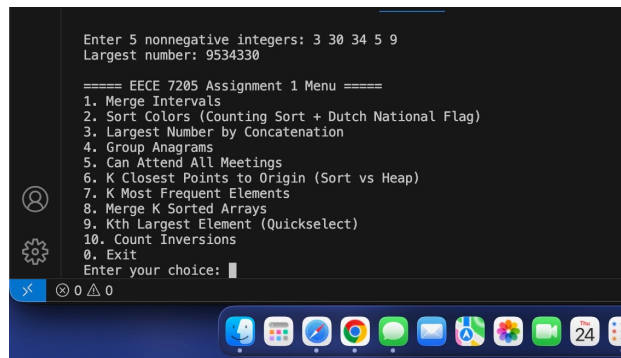


Figure 7: Test Case 1

```
Enter 2 nonnegative integers: 0 0
Largest number: 0

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points to Origin (Sort vs Heap)
7. K Most Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element (Quickselect)
10. Count Inversions
0. Exit
Enter your choice: █
```

Figure 8: Test Edge Case 2: All Zeros

```
Enter 2 nonnegative integers: 10 2
Largest number: 210

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points to Origin (Sort vs Heap)
7. K Most Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element (Quickselect)
10. Count Inversions
0. Exit
Enter your choice: █
```

Figure 9: Test Edge Case 3: Different Lengths

Problem 4: Group Anagrams

1. Understanding the Problem

Description: Anagrams contain the same characters with the same frequencies but in a different order. Grouping them is useful in text indexing and search. Given a vector of strings, group the anagrams together. The order of groups does not matter, but words within each group should retain their original input order.

Goal: Group strings that are anagrams of each other into the same group.

Inputs: Vector of strings.

Outputs: Collection of groups, in which each group has strings that are anagrams of each other. Words in each group should keep their original relative order.

Constraints: One constraint is that strings contain lowercase letters.

Assumptions: One assumption is that group order doesn't matter. However, order within the group does matter.

Edge Cases:

1. Empty input list
2. String with no anagram matches
3. Duplicate strings
4. Empty input string

2. Designing the Solution

For this problem I used a hash map where the key is the sorted version of each string, and the value is a list of all original strings that share that sorted form. Two strings are anagrams if and only if sorting their letters gives the same result, so this key naturally groups anagrams together. A hash map is appropriate here because it lets me group strings in one pass without comparing every pair directly.

3. Pseudocode

Algorithm 5 Group Anagrams

```
1: function GROUPANAGRAMS(strs)
2:   map ← empty hash map from string to list of strings
3:   for each word in strs do
4:     key ← word with its characters sorted
5:     add word to map[key]
6:   end for
7:   result ← empty list
8:   for each list group in map do
9:     add group to result
10:  end for
11:  return result
12: end function
```

4. Correctness Explanation

Sorting the characters of a word gives a new structured form. Two words are anagrams exactly when their sorted forms match, so grouping by this key is guaranteed to put anagrams together and keep non-anagrams apart. Since words are processed in input order and each is appended to its key's list, the original relative order within each group is preserved. Duplicate strings are tolerated: they sort to the same key and just get added to the same group, which is correct since a word is trivially an anagram of itself. An empty input produces an empty map, so the result is just an empty list. An empty string sorts to itself without problems. Because the hash map does not keep insertion order, the groups may print in a different order each time, which the problem allows.

5. Time and Space Complexity

For time complexity, sorting each string takes $O(k \log k)$, done for all n strings, giving $O(n \cdot k \log k)$ time. This is the same in the best, average, and worst case since every string must be sorted regardless of input arrangement. Space complexity is $O(n \cdot k)$ to store all the strings in the map.

6. C++ Implementation

```
1 vector<vector<string>> groupAnagrams(vector<string>& strs) {
2     unordered_map<string, vector<string>> map;
3
4     for (const string& word : strs) {
5         string key = word;
6         sort(key.begin(), key.end());
7         map[key].push_back(word);
8     }
9
10    vector<vector<string>> result;
11    for (auto& pair : map) {
12        result.push_back(pair.second);
13    }
14    return result;
15 }
16
17 void runProblem4() {
18     int n;
19     cout << "Enter number of strings: ";
20     cin >> n;
21
22     vector<string> strs(n);
23     cout << "Enter " << n << " strings: ";
24     for (int i = 0; i < n; i++) {
25         cin >> strs[i];
26     }
27
28     vector<vector<string>> groups = groupAnagrams(strs);
29
30     cout << "Groups: " << endl;
31     for (auto& group : groups) {
32         cout << "[" << " ";
33         for (auto& word : group) cout << word << " ";
34         cout << "]" << endl;
35     }
36 }
```

```

0. Exit
Enter your choice: 4
Enter number of strings: 6
Enter 6 strings: eat tea tan ate nat bat
Groups:
[ bat ]
[ tan nat ]
[ eat tea ate ]

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points to Origin (Sort vs Heap)

```

Figure 10: Test Case 1

```

0. Exit
Enter your choice: 4
Enter number of strings: 0
Enter 0 strings: Groups:

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points to Origin (Sort vs Heap)
7. K Most Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element (Quickselect)
10. Count Inversions

```

Figure 11: Test Edge Case 2: Empty Input

7. Test Cases

Test	Input	Expected Output
1 (given example)	eat tea tan ate nat bat	[eat, tea, ate] [tan, nat] [bat] (groups may appear in any order)
2 (edge case: duplicate strings)	ab ba ab	[ab, ba, ab]
3 (edge case: empty input)	(none)	(empty)

8. Execution Screenshots

```

10. Count Inversions
0. Exit
Enter your choice: 4
Enter number of strings: 3
Enter 3 strings: ab ba ab
Groups:
[ ab ba ab ]

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points to Origin (Sort vs Heap)
7. K Most Frequent Elements

```

Figure 12: Test Edge Case 3: Duplicate Strings

Problem 5: Meeting Rooms

1. Understanding the Problem

Description: Calendar software must determine whether a person or resource has been assigned to overlapping meetings. Given meeting intervals $[start, end]$, determine whether one person can attend every meeting. Meetings that begin exactly when another ends are allowed.

Goal: The goal here is to determine whether a person can attend every meeting, meaning no two meetings overlap.

Inputs: A vector of intervals from start to end.

Outputs: A boolean: **true** if no meetings overlap, **false** otherwise.

Constraints: One constraint is $start \leq end$ for each meeting. Meetings that touch exactly at an endpoint are allowed.

Assumptions: Meetings are not necessarily given in sorted order.

Edge Cases:

1. Empty input
2. Single meeting, no overlap
3. Two meetings that touch exactly at an endpoint
4. Two meetings that overlap slightly

2. Designing the Solution

For my solution, I sorted the meetings by start time, then checked each consecutive pair. If a meeting starts before the previous one ends, there's a conflict. Sorting first works because after sorting, the only meeting that could possibly overlap with the current one is the one right before it. Anything earlier has already ended before this one started, otherwise it would have been caught in an earlier comparison.

3. Pseudocode

Algorithm 6 Can Attend Meetings

```
1: function CANATTENDMEETINGS(intervals)
2:   if intervals has 0 or 1 elements then
3:     return true
4:   end if
5:   sort intervals by starting value
6:   for  $i \leftarrow 1$  to LENGTH(intervals) - 1 do
7:     if intervals[ $i$ ].start < intervals[ $i - 1$ ].end then
8:       return false
9:     end if
10:  end for
11:  return true
12: end function
```

4. Correctness Explanation

After sorting by start time, two meetings can only overlap if they are adjacent in the sorted order, and if meeting i doesn't overlap meeting $i - 1$, it can't overlap anything earlier either, since those all ended even sooner. So checking each consecutive pair is enough to catch every possible conflict. The condition uses strict $<$ rather than $\leq$, so meetings that touch exactly at an endpoint are allowed. An empty list or a single meeting has no pairs to check, so it correctly returns true.

5. Time and Space Complexity

Sorting takes $O(n \log n)$, which dominates the $O(n)$ pass that checks for overlaps, so the total time is $O(n \log n)$ in the average and worst case. In the best case, with 0 or 1 meetings, the function returns immediately in $O(1)$. The meetings are sorted in place, so the only additional space is the $O(\log n)$ used internally by `std::sort`.

6. C++ Implementation

```
1 bool canAttendMeetings(vector<vector<int>>& intervals) {
2     if (intervals.size() <= 1) return true;
3
4     sort(intervals.begin(), intervals.end(),
5         [](const vector<int>& a, const vector<int>& b) {
6             return a[0] < b[0];
7         });
8
9     for (size_t i = 1; i < intervals.size(); i++) {
10        if (intervals[i][0] < intervals[i - 1][1]) {
11            return false;
12        }
13    }
14    return true;
15 }
16
17 void runProblem5() {
18     int n;
19     cout << "Enter number of meetings: ";
20     cin >> n;
21
22     vector<vector<int>> intervals(n, vector<int>(2));
23     for (int i = 0; i < n; i++) {
24         cout << "Enter start and end for meeting " << i + 1 << ": ";
25         cin >> intervals[i][0] >> intervals[i][1];
26     }
27
28     bool result = canAttendMeetings(intervals);
29     cout << "Can attend all meetings: " << (result ? "true" : "false") << endl;
30 }
```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Enter start and end for meeting 3: 15 20
Can attend all meetings: false

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points to Origin (Sort vs Heap)
7. K Most Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element (Quickselect)
10. Count Inversions
0. Exit
Enter your choice: █

```

Figure 13: Test Case 1: Returns False If Overlap

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Enter start and end for meeting 2: 2 4
Can attend all meetings: true

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points to Origin (Sort vs Heap)
7. K Most Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element (Quickselect)
10. Count Inversions
0. Exit
Enter your choice: █

```

Figure 14: Test Edge Case 2: Returns True If No Overlap

7. Test Cases

Test	Input	Expected Output
1 (given example)	[0, 30] [5, 10] [15, 20]	false
2 (no overlap)	[7, 10] [2, 4]	true
3 (edge case: empty input)	(none)	true

8. Execution Screenshots

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Enter number of meetings: 0
Can attend all meetings: true

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points to Origin (Sort vs Heap)
7. K Most Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element (Quickselect)
10. Count Inversions
0. Exit
Enter your choice: █

```

Figure 15: Test Edge Case 3: Empty Input

Problem 6: K Closest Points to the Origin

1. Understanding the Problem

Description: Navigation, robotics, and spatial systems often need to identify the nearest locations or sensor readings. Given points (x, y) and an integer k , return the k points closest to the origin. Compare a sorting solution with a heap-based solution.

Goal: In this problem the goal is to find the k points closest to the origin $(0, 0)$ out of a given list of points.

Inputs: A vector of points (x, y) and an integer k .

Outputs: A vector containing the k closest points to the origin, from both the sorting and heap approaches. The program shows each point's squared distance, the time each approach took, and whether both approaches found the same distances.

Constraints: $1 \leq k \leq \text{number of points}$. The program asks again if fewer than 1 point or a k outside this range is entered.

Assumptions: One assumption is that distance is measured using standard Euclidean distance; since we only compare distances, squared distance can be used to avoid unnecessary square root calculations. The squared distance is stored as a `long long` so large coordinates do not overflow.

Edge Cases:

1. k equals total number of points
2. Points with equal distance from origin
3. Point exactly at origin of $(0, 0)$
4. Single point input with $k = 1$

2. Designing the Solution

For this problem I implemented two approaches, the sorting and heap-based approaches. For sorting, the program computes each point's squared distance from the origin, sorts all points by that distance, and takes the first k . For the heap approach, the program maintains a max-heap of size k . For each point, push it onto the heap; if the heap grows past size k , pop the farthest point. At the end, the heap contains exactly the k closest points, which are listed closest first. The sorting approach is simple and efficient to implement, but it sorts the entire list even though we only need the top k . The heap approach is more efficient when k is much smaller than n , because it only ever keeps k elements in memory and never needs to fully sort the whole array. To compare the two, the program times each approach in microseconds and checks that both returned points with the same distances.

3. Pseudocode

Algorithm 7 K Closest Points (Sorting)

```
1: function KCLOSESTSORT(points, k)
2:   sort points by  $x^2 + y^2$  from smallest to largest
3:   result  $\leftarrow$  empty list
4:   for  $i \leftarrow 0$  to  $k - 1$  do
5:     add points[i] to result
6:   end for
7:   return result
8: end function
```

Algorithm 8 K Closest Points (Heap)

```
1: function KCLOSESTHEAP(points, k)
2:   maxHeap  $\leftarrow$  empty max-heap ordered by  $x^2 + y^2$ 
3:   for each point p in points do
4:     push p onto maxHeap
5:     if size of maxHeap  $> k$  then
6:       pop the farthest point from maxHeap
7:     end if
8:   end for
9:   result  $\leftarrow$  empty list
10:  while maxHeap is not empty do
11:    add top of maxHeap to result
12:    pop maxHeap
13:  end while
14:  reverse result
15:  return result
16: end function
```

4. Correctness Explanation

For the sorting approach, all points are sorted by distance from smallest to largest, so the first k points in that order are the k closest points. For the heap approach, the max heap always holds the k smallest distance points seen so far. Whenever a new point is added and the heap exceeds size k , removing the point with the largest distance ensures only the closest k remain. By the end every point has been considered and the heap can only contain the true k closest points. For edge cases, points with the same coordinates are treated as separate points and compared purely by distance, so duplicates are handled the same as any other point. If two points are equally distant from the origin, either may be included depending on processing order, but this algorithm tolerates this because it doesn't require a tie breaking rule. For a point at the origin, it also correctly computes a distance of 0, making it automatically one of the closest points. If k equals the number of points, both algorithms naturally return every point in this scenario.

5. Time and Space Complexity

For the sorting approach, the time complexity is $O(n \log n)$ for the sort, the same in best, average, and worst case, and the space complexity is $O(n)$, since the points are copied before sorting. For the heap approach, $O(n \log k)$ is the time complexity, since each of the n points is pushed/popped from a heap of size at most k , and each heap operation costs $O(\log k)$. This is the same across best, average, and worst cases. The space complexity of the heap approach is $O(k)$, since the heap never holds more than k elements. For the small test inputs below, both approaches only take a few microseconds, and the times vary from run to run; the heap's advantage shows up when n is large and k is small.

6. C++ Implementation

```
1 long long squaredDist(const vector<int>& p) {
2     long long x = p[0];
3     long long y = p[1];
4     return x * x + y * y;
5 }
6
7 bool closerToOrigin(const vector<int>& a, const vector<int>& b) {
8     return squaredDist(a) < squaredDist(b);
9 }
10
11 vector<vector<int>> kClosestSort(vector<vector<int>> points, int k) {
12     sort(points.begin(), points.end(), closerToOrigin);
13
14     vector<vector<int>> result;
15     for (int i = 0; i < k; i++) {
16         result.push_back(points[i]);
17     }
18     return result;
19 }
20
21 vector<vector<int>> kClosestHeap(const vector<vector<int>>& points, int k) {
22     priority_queue<vector<int>, vector<vector<int>>, bool (*)(const vector<int>&, const vector<int>&)>
23         maxHeap(closerToOrigin);
24
25     for (const vector<int>& p : points) {
26         maxHeap.push(p);
27         if ((int)maxHeap.size() > k) {
28             maxHeap.pop();
29         }
30     }
31
32     vector<vector<int>> result;
33     while (!maxHeap.empty()) {
34         result.push_back(maxHeap.top());
35         maxHeap.pop();
36     }
37     reverse(result.begin(), result.end());
38     return result;
39 }
40
41 void printPoints(const vector<vector<int>>& points) {
42     for (const vector<int>& p : points) {
```

```

43     cout << "(" << p[0] << "," << p[1] << ") ";
44 }
45 cout << endl;
46 }
47
48 void printPointsWithDistance(const vector<vector<int>>& points) {
49     for (const vector<int>& p : points) {
50         cout << " (" << p[0] << "," << p[1] << ") squared distance = "
51             << squaredDist(p) << endl;
52     }
53 }
54
55 bool sameDistances(const vector<vector<int>>& a, const vector<vector<int>>& b) {
56     if (a.size() != b.size()) return false;
57     for (int i = 0; i < (int)a.size(); i++) {
58         if (squaredDist(a[i]) != squaredDist(b[i])) return false;
59     }
60     return true;
61 }
62
63 void runProblem6() {
64     int n, k;
65     cout << "Enter number of points: ";
66     cin >> n;
67     while (n < 1) {
68         cout << "Please enter at least 1 point: ";
69         cin >> n;
70     }
71
72     vector<vector<int>> points(n, vector<int>(2));
73     for (int i = 0; i < n; i++) {
74         cout << "Enter x and y for point " << i + 1 << ": ";
75         cin >> points[i][0] >> points[i][1];
76     }
77
78     cout << "Enter k (1 to " << n << "): ";
79     cin >> k;
80     while (k < 1 || k > n) {
81         cout << "k must be between 1 and " << n << ". Enter k: ";
82         cin >> k;
83     }
84
85     auto sortStart = chrono::high_resolution_clock::now();
86     vector<vector<int>> sortResult = kClosestSort(points, k);
87     auto sortEnd = chrono::high_resolution_clock::now();
88
89     auto heapStart = chrono::high_resolution_clock::now();
90     vector<vector<int>> heapResult = kClosestHeap(points, k);
91     auto heapEnd = chrono::high_resolution_clock::now();
92
93     long long sortTime = chrono::duration_cast<chrono::microseconds>(sortEnd - sortStart).count();
94     long long heapTime = chrono::duration_cast<chrono::microseconds>(heapEnd - heapStart).count();
95
96     cout << "Sorting approach result: ";
97     printPoints(sortResult);
98     printPointsWithDistance(sortResult);
99
100     cout << "Heap approach result: ";
101     printPoints(heapResult);

```

```

Enter x and y for point 2: -2 2
Enter k (1 to 2): 1
Sorting approach result: (-2,2)
(-2,2) squared distance = 8
Heap approach result: (-2,2)
(-2,2) squared distance = 8

Comparison:
Sorting approach time: 1374 microseconds (O(n log n))
Heap approach time: 330 microseconds (O(n log k))
Same distances found: yes

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation

```

Figure 16: Test Case 1: Closest Point to Origin

```

Enter k (1 to 2): 2
Sorting approach result: (0,0) (3,4)
(0,0) squared distance = 0
(3,4) squared distance = 25
Heap approach result: (0,0) (3,4)
(0,0) squared distance = 0
(3,4) squared distance = 25

Comparison:
Sorting approach time: 1226 microseconds (O(n log n))
Heap approach time: 31 microseconds (O(n log k))
Same distances found: yes

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)

```

Figure 17: Test Case 2: K=N Point of Origin

```

102 printPointsWithDistance(heapResult);
103
104 cout << "\nComparison:" << endl;
105 cout << "  Sorting approach time: " << sortTime << " microseconds (O(n log n))" << endl;
106 cout << "  Heap approach time:      " << heapTime << " microseconds (O(n log k))" << endl;
107 cout << "  Same distances found:    " << (sameDistances(sortResult, heapResult) ? "yes" : "no")
108 << endl;
    }

```

7. Test Cases

Test	Input	Expected Output
1 (given example)	(1, 3) (-2, 2), $k = 1$	(-2, 2)
2 (edge case: k equals total points, point at origin)	(0, 0) (3, 4), $k = 2$	(0, 0) (3, 4)
3 (edge case: single point)	(1, 1), $k = 1$	(1, 1)

8. Execution Screenshots

```
10. Count Inversions
0. Exit
Enter your choice: 6
Enter number of points: 1
Enter x and y for point 1: 1 1
Enter k (1 to 1): 1
Sorting approach result: (1,1)
(1,1) squared distance = 2
Heap approach result: (1,1)
(1,1) squared distance = 2

Comparison:
Sorting approach time: 1167 microseconds ( $O(n \log n)$ )
Heap approach time: 388 microseconds ( $O(n \log k)$ )
Same distances found: yes
```

Figure 18: Test Edge Case 3: Single Point

Problem 7: Top K Frequent Values

1. Understanding the Problem

Description: Monitoring and analytics systems often rank events, error codes, or queries by frequency.

Question: Given an integer array and an integer k , return the k most frequent values. If frequencies tie, return the smaller value first.

Goal: In this problem the goal is to find the k most frequent values in an integer array. If two values have the same frequency, the smaller value should be returned first.

Inputs: An integer array *nums* and an integer k .

Outputs: A vector of the k most frequent values, ordered by frequency descending, with ties broken by smaller values first. The program also prints a frequency table marking the selected values, explains any tie at the cutoff, and shows each selected value's count.

Constraints: $1 \leq k \leq$ number of distinct values in *nums*. The program asks again if the array is empty or k is outside this range.

Assumptions: One assumption is that the array can contain negative numbers, duplicates, and any range of integer values.

Edge Cases:

1. k equals number of distinct values
2. All elements are same value
3. Multiple values tied at same frequency
4. Single element array

2. Designing the Solution

I first counted the frequency of each value using a hash map. Then I sorted the distinct values with a custom comparator: higher frequency comes first, and if frequencies are equal, the smaller value comes first. Finally, I took the first k values from that sorted list. A hash map fits well for counting since it gives $O(1)$ average lookups/updates, and sorting the pairs directly handles the tie-breaking rule cleanly.

3. Pseudocode

Algorithm 9 K Most Frequent

```
1: function KMOSTFREQUENT(nums, k)
2:   freq  $\leftarrow$  empty hash map from value to count
3:   for each num in nums do
4:     freq[num]  $\leftarrow$  freq[num] + 1
5:   end for
6:   pairs  $\leftarrow$  list of (value, count) from freq
7:   sort pairs so that a comes before b if a.count > b.count,
     or if a.count = b.count and a.value < b.value
8:   result  $\leftarrow$  empty list
9:   for i  $\leftarrow$  0 to k - 1 do
10:    add pairs[i].value to result
11:  end for
12:  return result
13: end function
```

4. Correctness Explanation

Counting frequencies with a hash map correctly tracks how often each distinct value appears, regardless of the order values appear in the input. Sorting by frequency descending places the most frequent values first, satisfying the main requirement. The tie-breaking rule in the comparator directly enforces the problem's tie condition, so any two values with the same frequency are always ordered correctly relative to each other. Taking the first k after this sort guarantees the correct set and correct order. For duplicates, this algorithm handles this with ease because duplicates just increase the count in the hash map and are treated as a single entry when sorting. For dealing with ties, the program tolerates this by the second condition within the comparator, and the tie explanation printed by the program shows which tied values were chosen. Because k must be at least 1 and at most the number of distinct values, the array always has at least one element.

5. Time and Space Complexity

Building the frequency map takes $O(n)$ time and sorting the d distinct values takes $O(d \log d)$. So the overall time complexity is $O(n + d \log d)$. In the best case, when all elements are the same value ($d = 1$), this is $O(n)$. In the worst case, when all elements are distinct ($d = n$), it is $O(n \log n)$. Space complexity is $O(d)$ for the hash map and the list of pairs.

6. C++ Implementation

```
1 unordered_map<int, int> countFrequencies(const vector<int>& nums) {
2     unordered_map<int, int> freq;
3     for (int num : nums) {
4         freq[num]++;
5     }
6     return freq;
7 }
8
9 bool higherFrequency(const pair<int,int>& a, const pair<int,int>& b) {
10     if (a.second != b.second) return a.second > b.second;
```

```

11     return a.first < b.first;
12 }
13
14 vector<pair<int,int>> sortByFrequency(const vector<int>& nums) {
15     unordered_map<int, int> freq = countFrequencies(nums);
16
17     vector<pair<int,int>> pairs(freq.begin(), freq.end());
18     sort(pairs.begin(), pairs.end(), higherFrequency);
19     return pairs;
20 }
21
22 vector<int> kMostFrequent(const vector<int>& nums, int k) {
23     vector<pair<int,int>> pairs = sortByFrequency(nums);
24
25     vector<int> result;
26     for (int i = 0; i < k && i < (int)pairs.size(); i++) {
27         result.push_back(pairs[i].first);
28     }
29     return result;
30 }
31
32 void printFrequencyTable(const vector<pair<int,int>>& pairs, int k) {
33     cout << "Frequency table (highest first, ties -> smaller value first):" << endl;
34     for (int i = 0; i < (int)pairs.size(); i++) {
35         cout << " " << pairs[i].first << " appears " << pairs[i].second
36             << (pairs[i].second == 1 ? " time" : " times");
37         if (i < k) cout << "  <- selected";
38         cout << endl;
39     }
40 }
41
42 void printTieExplanation(const vector<pair<int,int>>& pairs, int k) {
43     if (k >= (int)pairs.size()) return;
44
45     int cutoffCount = pairs[k - 1].second;
46     if (pairs[k].second != cutoffCount) {
47         cout << "No tie at the cutoff." << endl;
48         return;
49     }
50
51     vector<int> chosen;
52     vector<int> leftOut;
53     for (int i = 0; i < (int)pairs.size(); i++) {
54         if (pairs[i].second == cutoffCount) {
55             if (i < k) chosen.push_back(pairs[i].first);
56             else leftOut.push_back(pairs[i].first);
57         }
58     }
59
60     cout << "Tie at the cutoff: these values all appear " << cutoffCount
61         << (cutoffCount == 1 ? " time." : " times.") << endl;
62     cout << " Chosen (smaller values): ";
63     for (int v : chosen) cout << v << " ";
64     cout << endl;
65     cout << " Left out (larger values): ";
66     for (int v : leftOut) cout << v << " ";
67     cout << endl;
68 }
69

```

```

70 void printResultWithCounts(const vector<int>& result, const vector<int>& nums) {
71     unordered_map<int, int> freq = countFrequencies(nums);
72     cout << "Top " << result.size() << " frequent values with counts: ";
73     for (int i = 0; i < (int)result.size(); i++) {
74         if (i > 0) cout << ", ";
75         cout << result[i] << " (" << freq[result[i]] << "x)";
76     }
77     cout << endl;
78 }
79
80 void runProblem7() {
81     int n, k;
82     cout << "Enter number of elements: ";
83     cin >> n;
84     while (n < 1) {
85         cout << "Please enter at least 1 element: ";
86         cin >> n;
87     }
88
89     vector<int> nums(n);
90     cout << "Enter " << n << " integers: ";
91     for (int i = 0; i < n; i++) {
92         cin >> nums[i];
93     }
94
95     int distinct = (int)countFrequencies(nums).size();
96     cout << "Enter k (1 to " << distinct << "): ";
97     cin >> k;
98     while (k < 1 || k > distinct) {
99         cout << "k must be between 1 and " << distinct << ". Enter k: ";
100        cin >> k;
101    }
102
103    cout << "Total elements: " << n << ", distinct values: " << distinct << endl;
104
105    vector<pair<int, int>> pairs = sortByFrequency(nums);
106    printFrequencyTable(pairs, k);
107    printTieExplanation(pairs, k);
108
109    vector<int> result = kMostFrequent(nums, k);
110    printResultWithCounts(result, nums);
111    cout << "Top " << k << " frequent values: ";
112    for (int v : result) cout << v << " ";
113    cout << endl;
114 }

```

7. Test Cases

Test	Input	Expected Output
1 (given example)	1 1 1 2 2 3, $k = 2$	1 2
2 (edge case: all frequencies tied)	1 2 3, $k = 3$	1 2 3
3 (edge case: single element)	5, $k = 1$	5

8. Execution Screenshots

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Enter number of elements: 6
Enter 6 integers: 1 1 1 2 2 3
Enter k (1 to 3): 2
Total elements: 6, distinct values: 3
Frequency table (highest first, ties -> smaller value first):
  1 appears 3 times  <- selected
  2 appears 2 times  <- selected
  3 appears 1 time
No tie at the cutoff.
Top 2 frequent values with counts: 1 (3x), 2 (2x)
Top 2 frequent values: 1 2

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation

```

Figure 19: Test Case 1

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

10. Count Inversions
0. Exit
Enter your choice: 7
Enter number of elements: 3
Enter 3 integers: 1 2 3
Enter k (1 to 3): 3
Total elements: 3, distinct values: 3
Frequency table (highest first, ties -> smaller value first):
  1 appears 1 time  <- selected
  2 appears 1 time  <- selected
  3 appears 1 time  <- selected
Top 3 frequent values with counts: 1 (1x), 2 (1x), 3 (1x)
Top 3 frequent values: 1 2 3

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals

```

Figure 20: Test Edge Case 2: All Frequencies Tied

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

10. Count Inversions
0. Exit
Enter your choice: 7
Enter number of elements: 1
Enter 1 integers: 5
Enter k (1 to 1): 1
Total elements: 1, distinct values: 1
Frequency table (highest first, ties -> smaller value first):
  5 appears 1 time  <- selected
Top 1 frequent values with counts: 5 (1x)
Top 1 frequent values: 5

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation

```

Figure 21: Test Edge Case 3: Single Element

Problem 8: Merge K Sorted Arrays

1. Understanding the Problem

Description: Distributed systems may produce multiple individually sorted streams that must be combined into one ordered result.

Question: Given k sorted integer arrays, merge them into one sorted array. Design a solution that does not repeatedly sort the complete output.

Goal: The goal here is to merge k sorted integer arrays into a single sorted array, without repeatedly sorting the entire combined output.

Inputs: A collection of k arrays, each individually sorted in ascending order. In the program, the user enters the number of arrays, then the length and values of each array.

Outputs: A single array containing all elements from the k arrays, in sorted order.

Constraints: Arrays may vary in length, and some may be empty. The program asks again if a count or length is negative, or if an entered array is not sorted.

Assumptions: Arrays are already sorted individually. Duplicate values across or within arrays are allowed.

Edge Cases:

1. $k = 0$
2. One or more empty arrays among k arrays
3. Only one array given
4. All arrays contain the same values

2. Designing the Solution

For this problem I used a min-heap that holds at most one element from each array at a time. Each heap entry is a struct that stores the value, which array it came from, and its position in that array. The algorithm starts by pushing the first element of every non-empty array onto the heap. It then repeatedly removes the smallest entry, adds it to the result, and pushes the next element from the same array. Because each array is already sorted, the smallest remaining element overall is always at the front of one of the arrays, so it is always in the heap. A min-heap fits this problem because it gives the smallest value in $O(\log k)$ time, and the output is built in order in one pass without ever sorting the combined output.

3. Pseudocode

Algorithm 10 Merge K Sorted Arrays

```
1: function MERGEKSORTEDARRAYS(arrays)
2:   minHeap  $\leftarrow$  empty min-heap of (value, arrayIndex, elementIndex)
3:   for i  $\leftarrow$  0 to k - 1 do
4:     if arrays[i] is not empty then
5:       push (arrays[i][0], i, 0) onto minHeap
6:     end if
7:   end for
8:   result  $\leftarrow$  empty list
9:   while minHeap is not empty do
10:    (value, a, e)  $\leftarrow$  pop minHeap
11:    add value to result
12:    if e + 1 < LENGTH(arrays[a]) then
13:      push (arrays[a][e + 1], a, e + 1) onto minHeap
14:    end if
15:  end while
16:  return result
17: end function
```

4. Correctness Explanation

The heap always holds the smallest not-yet-used element of every array that still has elements left. Since each array is sorted, that element is at the front of the array, so the top of the heap is the smallest remaining element overall. Adding it to the result keeps the result sorted, and pushing the next element from the same array keeps the heap complete. Every element is pushed and popped exactly once, so the result contains all of the elements. Duplicate values are tolerated, since equal values can be added in either order. Empty arrays are never pushed onto the heap, so they are skipped. When $k = 0$ or all arrays are empty, the heap starts empty and the result is an empty array.

5. Time and Space Complexity

Let N be the total number of elements across all arrays. Each element is pushed and popped once on a heap that holds at most k entries, and each heap operation costs $O(\log k)$, so the time complexity is $O(N \log k)$. This is the same in the best, average, and worst case. The space complexity is $O(k)$ for the heap, plus $O(N)$ for the result array.

6. C++ Implementation

```
1 struct HeapEntry {
2     int value;
3     int arrIdx;
4     int elIdx;
5 };
6
7 struct CompareEntry {
8     bool operator()(const HeapEntry& a, const HeapEntry& b) {
```

```

9         return a.value > b.value;
10     }
11 };
12
13 vector<int> mergeKSortedArrays(const vector<vector<int>>& arrays) {
14     priority_queue<HeapEntry, vector<HeapEntry>, CompareEntry> minHeap;
15
16     for (int i = 0; i < (int)arrays.size(); i++) {
17         if (!arrays[i].empty()) {
18             minHeap.push({arrays[i][0], i, 0});
19         }
20     }
21
22     vector<int> result;
23     while (!minHeap.empty()) {
24         HeapEntry top = minHeap.top();
25         minHeap.pop();
26         result.push_back(top.value);
27
28         if (top.elIdx + 1 < (int)arrays[top.arrIdx].size()) {
29             minHeap.push({arrays[top.arrIdx][top.elIdx + 1], top.arrIdx, top.elIdx + 1});
30         }
31     }
32     return result;
33 }
34
35 bool isSortedArray(const vector<int>& arr) {
36     for (int i = 1; i < (int)arr.size(); i++) {
37         if (arr[i] < arr[i - 1]) return false;
38     }
39     return true;
40 }
41
42 void printArray(const vector<int>& arr) {
43     if (arr.empty()) {
44         cout << "(empty)";
45         return;
46     }
47     for (int value : arr) cout << value << " ";
48 }
49
50 void runProblem8() {
51     int k;
52     cout << "Enter number of arrays: ";
53     cin >> k;
54     while (k < 0) {
55         cout << "Number of arrays cannot be negative. Enter number of arrays: ";
56         cin >> k;
57     }
58
59     vector<vector<int>> arrays(k);
60     for (int i = 0; i < k; i++) {
61         int len;
62         cout << "Enter length of array " << i + 1 << ": ";
63         cin >> len;
64         while (len < 0) {
65             cout << "Length cannot be negative. Enter length of array " << i + 1 << ": ";
66             cin >> len;
67         }

```

```

0. Exit
Enter your choice: 8
Enter number of arrays: 3
Enter length of array 1: 3
Enter 3 sorted values: 1 4 5
Enter length of array 2: 3
Enter 3 sorted values: 1 3 4
Enter length of array 3: 2
Enter 2 sorted values: 2 6
Merged sorted array: 1 1 2 3 4 4 5 6

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams

```

Figure 22: Test Case 1

```

68     arrays[i].resize(len);
69     cout << "Enter " << len << " sorted values: ";
70     for (int j = 0; j < len; j++) {
71         cin >> arrays[i][j];
72     }
73
74
75     while (!isSortedArray(arrays[i])) {
76         cout << "Array " << i + 1 << " is not sorted. Enter " << len << " sorted values: ";
77         for (int j = 0; j < len; j++) {
78             cin >> arrays[i][j];
79         }
80     }
81 }
82
83 vector<int> result = mergeKSortedArrays(arrays);
84
85 cout << "Merged sorted array: ";
86 printArray(result);
87 cout << endl;
88 }

```

7. Test Cases

Test	Input	Expected Output
1 (given example)	[1, 4, 5] [1, 3, 4] [2, 6]	1 1 2 3 4 4 5 6
2 (edge case: one empty array)	[] [1, 2, 3]	1 2 3
3 (edge case: no arrays, $k = 0$)	(none)	(empty)

8. Execution Screenshots

```
Enter 3 sorted values: 1 2 3
Merged sorted array: 1 2 3

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points to Origin (Sort vs Heap)
7. K Most Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element (Quickselect)
10. Count Inversions
0. Exit
Enter your choice: █
```

Figure 23: Test Edge Case 2: One Empty Array

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Enter number of arrays: 0
Merged sorted array: (empty)

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points to Origin (Sort vs Heap)
7. K Most Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element (Quickselect)
10. Count Inversions
0. Exit
Enter your choice: █
```

Figure 24: Test Edge Case 3: No Arrays

Problem 9: Kth Largest Element

1. Understanding the Problem

Description: Ranking systems often need one order statistic without paying the cost of fully sorting a large dataset.

Question: Given an unsorted integer array and k , return the k th-largest element. Duplicate values count as separate positions.

Goal: The goal here is to find the k th-largest element in an unsorted array without fully sorting it.

Inputs: An unsorted integer array and an integer k .

Outputs: The k th-largest element of the array.

Constraints: $1 \leq k \leq$ number of elements, and the array has at least one element. The program asks again if these are not met.

Assumptions: Duplicate values count as separate positions. For example, in 3 2 3 1 2 4 5 5 6, the values from largest to smallest are 6 5 5 4 ..., so the 4th largest is 4. The array can contain negative numbers.

Edge Cases:

1. Single element array
2. All values are equal
3. $k = 1$ (the largest value)
4. $k = n$ (the smallest value)
5. Duplicate values

2. Designing the Solution

For this problem I used quickselect. The k th-largest element is the element that would be at index $n - k$ if the array were sorted in ascending order. The partition step picks a pivot, moves every value less than or equal to the pivot to its left, and returns the pivot's final index. If that index is the target, the pivot is the answer. Otherwise, the algorithm only continues on the side that contains the target. Unlike quicksort, which sorts both sides, quickselect only works on one side, which is why it does not need to fully sort the array. I used the middle element as the pivot (swapping it to the end before partitioning) so that an already sorted array does not cause the worst case. The function works on a copy of the array so the original input is not rearranged.

3. Pseudocode

Algorithm 11 Partition

```
1: function PARTITION(nums, left, right)
2:   middle  $\leftarrow$  left + (right - left)/2
3:   swap nums[middle] and nums[right]
4:   pivot  $\leftarrow$  nums[right]
5:   i  $\leftarrow$  left
6:   for j  $\leftarrow$  left to right - 1 do
7:     if nums[j]  $\leq$  pivot then
8:       swap nums[i] and nums[j]
9:       i  $\leftarrow$  i + 1
10:    end if
11:  end for
12:  swap nums[i] and nums[right]
13:  return i
14: end function
```

Algorithm 12 Quickselect

```
1: function QUICKSELECT(nums, left, right, target)
2:   if left = right then
3:     return nums[left]
4:   end if
5:   pivotIndex  $\leftarrow$  PARTITION(nums, left, right)
6:   if pivotIndex = target then
7:     return nums[pivotIndex]
8:   else if target < pivotIndex then
9:     return QUICKSELECT(nums, left, pivotIndex - 1, target)
10:  else
11:    return QUICKSELECT(nums, pivotIndex + 1, right, target)
12:  end if
13: end function
```

Algorithm 13 Find Kth Largest

```
1: function FINDKTHLARGEST(nums, k)
2:   target  $\leftarrow$  LENGTH(nums) - k
3:   return QUICKSELECT(copy of nums, 0, LENGTH(nums) - 1, target)
4: end function
```

4. Correctness Explanation

After partitioning, every element to the left of the pivot is less than or equal to it and every element to the right is greater, so the pivot ends up in exactly the position it would have in sorted order. If that position is the target index, the pivot is the answer. If not, the answer must be on the side that contains the target, so the other side can be safely ignored. Each call makes the range smaller by at

least one element, so the algorithm always finishes. Duplicate values each take their own position after partitioning, which matches the requirement that duplicates count as separate positions. For a single element array, *left* equals *right* and that element is returned.

5. Time and Space Complexity

The average time complexity is $O(n)$, because each partition cuts the search range down by about half on average, so the total work is about $n + n/2 + n/4 + \dots \leq 2n$. The best case is also $O(n)$, since the first partition always looks at every element. The worst case is $O(n^2)$, which happens if the pivot is repeatedly the smallest or largest value in the range; using the middle element avoids this for sorted input, but it can still happen with many repeated values or specially arranged input. The space complexity is $O(n)$ for the copy of the array, plus the recursion stack, which is $O(\log n)$ on average and $O(n)$ in the worst case.

6. C++ Implementation

```

1  int partition(vector<int>& nums, int left, int right) {
2      int middle = left + (right - left) / 2;
3      swap(nums[middle], nums[right]);
4
5      int pivot = nums[right];
6      int i = left;
7      for (int j = left; j < right; j++) {
8          if (nums[j] <= pivot) {
9              swap(nums[i], nums[j]);
10             i++;
11         }
12     }
13     swap(nums[i], nums[right]);
14     return i;
15 }
16
17 int quickSelect(vector<int>& nums, int left, int right, int target) {
18     if (left == right) return nums[left];
19
20     int pivotIndex = partition(nums, left, right);
21
22     if (pivotIndex == target) {
23         return nums[pivotIndex];
24     } else if (target < pivotIndex) {
25         return quickSelect(nums, left, pivotIndex - 1, target);
26     } else {
27         return quickSelect(nums, pivotIndex + 1, right, target);
28     }
29 }
30
31 int findKthLargest(vector<int> nums, int k) {
32     int target = (int)nums.size() - k;
33     return quickSelect(nums, 0, (int)nums.size() - 1, target);
34 }
35
36 void runProblem9() {
37     int n, k;
38     cout << "Enter number of elements: ";
39     cin >> n;

```

```

8. Merge K Sorted Arrays
9. Kth Largest Element (Quickselect)
10. Count Inversions
0. Exit
Enter your choice: 9
Enter number of elements: 6
Enter 6 integers: 3 2 1 5 6 4
Enter k (1 to 6): 2
The kth largest element (k = 2) is: 5

==== EECE 7205 Assignment 1 Menu ====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings

```

Figure 25: Test Case 1

```

40 while (n < 1) {
41     cout << "Please enter at least 1 element: ";
42     cin >> n;
43 }
44
45 vector<int> nums(n);
46 cout << "Enter " << n << " integers: ";
47 for (int i = 0; i < n; i++) {
48     cin >> nums[i];
49 }
50
51 cout << "Enter k (1 to " << n << "): ";
52 cin >> k;
53 while (k < 1 || k > n) {
54     cout << "k must be between 1 and " << n << ". Enter k: ";
55     cin >> k;
56 }
57
58 int result = findKthLargest(nums, k);
59 cout << "The kth largest element (k = " << k << ") is: " << result << endl;
60 }

```

7. Test Cases

Test	Input	Expected Output
1 (given example)	3 2 1 5 6 4, $k = 2$	5
2 (edge case: duplicates count separately)	3 2 3 1 2 4 5 5 6, $k = 4$	4
3 (edge case: single element)	7, $k = 1$	7

8. Execution Screenshots

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

8. Merge K Sorted Arrays
9. Kth Largest Element (Quickselect)
10. Count Inversions
0. Exit
Enter your choice: 9
Enter number of elements: 9
Enter 9 integers: 3 2 3 1 2 4 5 5 6
Enter k (1 to 9): 4
The kth largest element (k = 4) is: 4

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
```

Figure 26: Test Edge Case 2: Duplicates Counted Separately

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

9. Kth Largest Element (Quickselect)
10. Count Inversions
0. Exit
Enter your choice: 9
Enter number of elements: 1
Enter 1 integers: 7
Enter k (1 to 1): 1
The kth largest element (k = 1) is: 7

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points to Origin (Sort vs Heap)
```

Figure 27: Test Edge Case 3: Single Element

Problem 10: Count Inversions

1. Understanding the Problem

Description: An inversion measures how far an array is from sorted order and is used in ranking analysis and sequence comparison.

Question: Count pairs (i, j) for which $i < j$ and $values[i] > values[j]$. Develop an efficient solution by modifying Merge Sort.

Goal: The goal here is to count every inversion in an array in $O(n \log n)$ time by modifying merge sort.

Inputs: An integer array, which may be empty.

Outputs: The number of inversions, stored as a 64-bit integer (`long long`).

Constraints: The solution must run in $O(n \log n)$ time and use a 64-bit integer for the count. The program asks again if a negative number of elements is entered.

Assumptions: Equal values are not inversions, since the definition uses strictly greater than. The array can contain negative numbers and duplicates.

Edge Cases:

1. Empty array
2. Single element
3. Array that is already sorted (0 inversions)
4. Array in reverse order (the most inversions possible, $n(n-1)/2$)
5. Duplicate values

2. Designing the Solution

For this problem I modified merge sort, since merge sort already compares elements from the two halves while it merges them. When merging a sorted left half and a sorted right half, if an element from the right half is smaller than the current element from the left half, it is also smaller than every remaining element in the left half, because the left half is sorted. So when that right element is taken, it adds $(mid - i + 1)$ inversions all at once instead of checking each pair one at a time. The total count is the inversions inside the left half, plus the inversions inside the right half, plus the inversions counted while merging. A temporary array is used for merging, and the count is stored as a `long long` because the largest possible count, $n(n-1)/2$, is too big for a regular `int` once n is larger than about 65,000.

3. Pseudocode

Algorithm 14 Count Inversions

```
1: function COUNTINVERSIONS(values)
2:   if values is empty then
3:     return 0
4:   end if
5:   temp  $\leftarrow$  array of LENGTH(values)
6:   return MERGESORTANDCOUNT(values, temp, 0, LENGTH(values) - 1)
7: end function
```

Algorithm 15 Merge Sort and Count

```
1: function MERGESORTANDCOUNT(values, temp, left, right)
2:   if left  $\geq$  right then
3:     return 0
4:   end if
5:   mid  $\leftarrow$  left + (right - left)/2
6:   count  $\leftarrow$  MERGESORTANDCOUNT(values, temp, left, mid)
7:   count  $\leftarrow$  count + MERGESORTANDCOUNT(values, temp, mid + 1, right)
8:   count  $\leftarrow$  count + MERGEANDCOUNT(values, temp, left, mid, right)
9:   return count
10: end function
```

Algorithm 16 Merge and Count

```
1: function MERGEANDCOUNT(values, temp, left, mid, right)
2:   i  $\leftarrow$  left
3:   j  $\leftarrow$  mid + 1
4:   k  $\leftarrow$  left
5:   count  $\leftarrow$  0
6:   while i  $\leq$  mid and j  $\leq$  right do
7:     if values[i]  $\leq$  values[j] then
8:       temp[k]  $\leftarrow$  values[i]
9:       i  $\leftarrow$  i + 1
10:    else
11:      temp[k]  $\leftarrow$  values[j]
12:      j  $\leftarrow$  j + 1
13:      count  $\leftarrow$  count + (mid - i + 1)
14:    end if
15:    k  $\leftarrow$  k + 1
16:  end while
17:  copy the remaining values[i..mid] into temp
18:  copy the remaining values[j..right] into temp
19:  copy temp[left..right] back into values
20:  return count
21: end function
```

4. Correctness Explanation

Every inversion either has both elements in the left half, both in the right half, or one in each half. The first two kinds are counted by the recursive calls. An inversion split across the halves pairs a left element with a smaller right element, and it is counted exactly once: when that right element is copied during the merge, the remaining left elements are exactly the ones greater than it, and there are $(mid - i + 1)$ of them. Using $\leq$ when taking from the left half means equal values are never counted. It is a recursion so it also sorts each half, which the merge step depends on. An empty array returns 0 right away, and a single element range returns 0.

5. Time and Space Complexity

The time complexity is $O(n \log n)$ in the best, average, and worst case. The recursion has $\log n$ levels, each level does $O(n)$ total work while merging, and this does not depend on the order of the input. The space complexity is $O(n)$ for the temporary array and the working copy of the input, plus $O(\log n)$ for the recursion stack.

6. C++ Implementation

```
1 long long mergeAndCount(vector<int>& values, vector<int>& temp, int left, int mid, int right) {
2     int i = left;
3     int j = mid + 1;
4     int k = left;
5     long long count = 0;
6
7     while (i <= mid && j <= right) {
8         if (values[i] <= values[j]) {
9             temp[k++] = values[i++];
10        } else {
11            temp[k++] = values[j++];
12            count += (mid - i + 1);
13        }
14    }
15
16    while (i <= mid) {
17        temp[k++] = values[i++];
18    }
19    while (j <= right) {
20        temp[k++] = values[j++];
21    }
22
23    for (int p = left; p <= right; p++) {
24        values[p] = temp[p];
25    }
26    return count;
27 }
28
29 long long mergeSortAndCount(vector<int>& values, vector<int>& temp, int left, int right) {
30     if (left >= right) return 0;
31
32     int mid = left + (right - left) / 2;
33     long long count = 0;
34     count += mergeSortAndCount(values, temp, left, mid);
35     count += mergeSortAndCount(values, temp, mid + 1, right);
36     count += mergeAndCount(values, temp, left, mid, right);
37 }
```

```

37     return count;
38 }
39
40 long long countInversions(vector<int> values) {
41     if (values.empty()) return 0;
42     vector<int> temp(values.size());
43     return mergeSortAndCount(values, temp, 0, (int)values.size() - 1);
44 }
45
46 void runProblem10() {
47     int n;
48     cout << "Enter number of elements: ";
49     cin >> n;
50     while (n < 0) {
51         cout << "Number of elements cannot be negative. Enter number of elements: ";
52         cin >> n;
53     }
54
55     vector<int> values(n);
56     if (n > 0) {
57         cout << "Enter " << n << " integers: ";
58     }
59     for (int i = 0; i < n; i++) {
60         cin >> values[i];
61     }
62
63     long long result = countInversions(values);
64     cout << "Number of inversions: " << result << endl;
65 }

```

7. Test Cases

Test	Input	Expected Output
1 (given example)	2 4 1 3 5	3 (inversions: (2, 1), (4, 1), (4, 3))
2 (edge case: reverse order)	5 4 3 2 1	10
3 (edge case: empty array)	(none)	0

8. Execution Screenshots

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

6. K Closest Points to Origin (Sort vs Heap)
7. K Most Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element (Quickselect)
10. Count Inversions
0. Exit
Enter your choice: 10
Enter number of elements: 5
Enter 5 integers: 2 4 1 3 5
Number of inversions: 3

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
```

Figure 28: Test Case 1

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

9. Kth Largest Element (Quickselect)
10. Count Inversions
0. Exit
Enter your choice: 10
Enter number of elements: 5
Enter 5 integers: 5 4 3 2 1
Number of inversions: 10

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points to Origin (Sort vs Heap)
7. K Most Frequent Elements
```

Figure 29: Test Case 2: Reverse Order

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

7. K Most Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element (Quickselect)
10. Count Inversions
0. Exit
Enter your choice: 10
Enter number of elements: 0
Number of inversions: 0

===== EECE 7205 Assignment 1 Menu =====
1. Merge Intervals
2. Sort Colors (Counting Sort + Dutch National Flag)
3. Largest Number by Concatenation
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points to Origin (Sort vs Heap)
```

Figure 30: Test Case 3: Empty Array